

AI 데이터 분석 · 9일차

튜플 활용과 셋

묶어서 지키고, 걸러서 비교한다

len · count · index · 튜플 리스트 · for 언패킹 · sorted · set · add · in · 집합 연산

교안 06_01(튜플과 셋) 후반부 + 종합 실습 1~3 정리

어제 만든 튜플을 실제로 다루고, 중복을 걸러내 두 목록을 비교하는 날

2026-07-31 AI 데이터 분석 교육 정리 (9일차)

학습자 Minho · 국립금오공과대학교 컴퓨터공학부

이 책을 읽는 법

어제는 튜플을 만드는 법까지 배웠습니다. 소괄호와 쉼표, 그리고 언패킹까지였죠. 오늘은 그 튜플을 실제로 다루는 날입니다. 개수를 세고, 위치를 찾고, 리스트에 모아 반복문으로 돌리고, 정렬합니다. 후반부에는 새 자료형 set이 등장해 중복을 걷어내고 두 목록을 비교합니다. 마지막에는 지금까지 배운 전부를 한 문제에 몰아넣은 종합 실습 3개를 풀니다.

튜플 len·count·index **패턴** 튜플 리스트 + for 언패킹 **내장함수** sorted **셋** set·add·in **연산자** | & -

연결

오늘의 주력 구조는 [(이름, 값), (이름, 값), ...] — **리스트 안에 튜플**입니다. 리스트(6일차) + 조건문·반복문(7일차) + while·필터링(8일차) + 튜플(8일차)이 여기서 한 줄로 합쳐집니다. 그리고 이 구조는 나중에 pandas의 DataFrame에 데이터를 넣을 때 그대로 쓰이는 형태입니다.

목차

	장 / STEP	무엇을 배우나
CHAPTER 0	오늘의 지도 — 묶기와 거르기	자료형 4형제 · 용어 3초 판별법
STEP 1	튜플을 다루는 세 도구	len · count · index
STEP 2	튜플 리스트와 for 언패킹	오늘의 주력 구조 · enumerate 결합
STEP 3	중첩 튜플과 sorted 정렬	메타데이터 · 정렬 기준 설계
STEP 4	셋 — 중복을 지우는 자료형	set() · 빈 셋 함정 · 종류 수
STEP 5	셋 다루기	add · in · sorted 변환
STEP 6	집합 연산 3종 세트	union · intersection · difference
STEP 7	교안 실습 1~6 복기	내가 제출한 코드 점검
STEP 8	종합 실습 1 — 모니터링 리포트	튜플 리스트 총동원
STEP 9	종합 실습 2 — 실시간 입력 시스템	while + 통계 · 버그 5종 해부
STEP 10	종합 실습 3 — 교대조 로그 분석	집합 연산 + 튜플 정렬
부록 A·B·C	치트시트 · 오류 사전 · 예고	한 장 요약 · 함정 모음

오늘의 지도 — 묶기와 거르기

네 자료형은 각자 다른 문제를 푼다

파이썬에 자료형이 여러 개인 이유는 **각자 잘하는 일이 다르기 때문**입니다. 리스트는 순서대로 쌓고 바꾸는 데 강하고, 튜플은 한 번 정한 짝을 지키는 데 강하고, 셋은 중복을 지우고 두 목록을 비교하는 데 강합니다. 오늘은 이 중 튜플의 활용법과 셋 전체를 배웁니다.

자료형	괄호	순서	중복	값 변경	대표 용도
리스트	[]	있음	허용	가능	계속 추가·수정되는 측정값
튜플	()	있음	허용	불가	바뀌면 안 되는 짝 (이름·값)
셋	{ }	없음	제거	추가·삭제	종류 세기 · 두 목록 비교
딕셔너리	{키:값}	있음	키 유일	가능	이름표로 찾기 — 다음 시간

개념

강사님 표현으로는 "**묶는 튜플, 거르는 셋, 찾는 딕셔너리**"입니다. 셋과 딕셔너리가 똑같이 중괄호 { }를 쓰는 것이 혼동 지점인데, 값만 나열하면 셋이고 키:값 형태면 딕셔너리입니다. 그래서 **빈 중괄호 {}**는 딕셔너리로 만들어집니다(STEP 4에서 다시 다룹니다).

용어 3초 판별법 — 오늘 배운 것에 적용하면

강사님이 오늘 여러 번 강조한 지점입니다. len은 **내장 함수**이고 count·index·add·union은 **메서드**입니다. 이 둘을 헷갈리면 status.len() 같은 존재하지 않는 코드를 쓰게 됩니다. 판별은 두 단계면 끝납니다.

① 앞에 점(.)이 있나?	② 뒤에 괄호()가 붙나?	결론	오늘의 예
없다	붙는다	내장 함수	len(t) · sorted(t) · set(t) · type(t)
있다	붙는다	메서드	t.count() · t.index() · s.add() · s.union()
있다	안 붙는다	속성	오늘은 등장하지 않음
없다	안 붙는다	키워드·연산자	in · · & · - · for · if

주의

`len`을 메서드로 착각하는 것이 초보 단계에서 가장 흔한 실수입니다. `len`은 영어 `length`의 줄임말이고, 파이썬이 **자료형과 무관하게** 길이를 재도록 만들어 둔 내장 함수라서 문자열·리스트·튜플·셋 어디에든 `len(대상)` 형태로 씁니다. 반대로 `count.index`는 그 자료형이 스스로 들고 있는 기능이라 `대상.count()`처럼 점을 찍습니다.

오늘의 흐름



왼쪽 두 칸이 오전, 오른쪽 세 칸이 오후입니다. 흐름을 한 문장으로 붙이면 **"짜지어진 데이터를 튜플로 묶어 리스트에 모으고, 반복문으로 처리한 뒤, 중복을 걸어내 두 목록을 비교한다"**가 됩니다. 이 문장이 오늘 하루의 요약입니다.

STEP 1

튜플을 다루는 세 도구

len · count · index

튜플은 값을 바꿀 수 없습니다. 그래서 리스트처럼 `append` · `remove` · `sort` 같은 수정용 메서드가 아예 없습니다. 튜플이 가진 메서드는 딱 두 개, `count`와 `index`뿐입니다. 여기에 내장 함수 `len`을 더한 셋이 튜플을 다루는 도구 전부입니다. 적어 보이지만 "몇 개인가 · 몇 번 나왔나 · 어디에 있나"라는 세 가지 질문을 모두 답할 수 있습니다.

내장함수 `len()` 전체 개수 세기

정의 괄호 안에 넣은 대상의 길이(요소 개수)를 정수로 돌려줍니다. `length`의 줄임말입니다.

형태 `len(대상)`

```
status = ("정상", "정상", "경고", "정상")
```

```
print(len(status))
```

```
# 리스트·문자열·셋에도 똑같이 쓸 수 있다
```

```
print(len([1, 2, 3]))
```

```
print(len("모터온도"))
```

```
▶ 4  
▶ 3  
▶ 4
```

왜 쓰나 데이터가 비어 있지 않은지 확인할 때 가장 먼저 씁니다. 개수를 모르는 채로 인덱스에 접근하면 `IndexError`가 나기 때문에, 처리 전에 `len`으로 확인하는 것이 습관이 되어야 합니다.

응용 평균을 낼 때 분모로 쓰는 것이 실전에서 가장 잦은 용도입니다. 그리고 `len(리스트)`와 `len(set(리스트))`를 비교하면 중복이 있는지를 한 줄로 알 수 있습니다 — 두 값이 다르면 중복이 있다는 뜻입니다.

```
logs = ["S01", "S02", "S01", "S03"]
```

```
print("기록 건수:", len(logs))
```

```
print("센서 종류:", len(set(logs)))
```

```
if len(logs) != len(set(logs)):  
    print("중복 기록 있음 — 정리 필요")
```

▶ 기록 건수: 4

▶ 센서 종류: 3

▶ 중복 기록 있음 — 정리 필요

주의

`len`은 내장 함수이므로 `status.len()`처럼 점을 찍어 쓰면 `AttributeError`가 납니다. 반드시 `len(status)` 형태입니다.

메서드 `.count()` 특정 값이 몇 번 나오는지 세기

정의 튜플 안에서 인자로 준 값이 몇 번 등장하는지 세어 정수로 돌려줍니다. 없으면 0입니다.

형태 튜플.`count(찾을_값)`

```
status = ("정상", "정상", "경고", "정상")
```

```
print(status.count("경고"))
```

```
print(status.count("정상"))
```

```
print(status.count("위험"))    # 없는 값
```

▶ 1

▶ 3

▶ 0

왜 쓰나 강사님이 든 예시가 그대로 실무입니다 — "경고가 몇 번 올랐는지"를 셀 때 씁니다. 상태 기록은 시간 순서 그대로 쌓이므로 정렬해서 세는 것이 아니라, 순서를 건드리지 않고 `count`로 세는 것이 맞습니다.

응용 `count`의 반환값이 정수라는 점을 이용하면 조건문에 바로 넣을 수 있습니다. 경고가 3회 이상이면 즉시 점검 대상으로 올리는 식의 판정 로직이 한 줄로 나옵니다.

```
status = ("정상", "경고", "경고", "정상", "경고")
LIMIT = 3
```

```
warn = status.count("경고")
print("경고 횟수:", warn, "/ 전체", len(status))
```

```
if warn >= LIMIT:
    print("경고 누적", warn, "회 — 정밀 점검 대상")
```

▶ 경고 횟수: 3 / 전체 5

▶ 경고 누적 3 회 — 정밀 점검 대상

자주 하는 실수

문자열은 **대소문자를 구분**합니다. `count("Warning")`과 `count("warning")`은 서로 다른 값을 셉니다. 실제로 수업 코드에서 `tup.count("warning")`은 2, `tup.count("Warning")`은 0이 나왔습니다.

메서드 `.index()` 특정 값이 처음 나온 위치 찾기

정의 인자로 준 값이 **처음 등장하는 인덱스 번호**를 돌려줍니다. 값이 없으면 오류가 납니다.

형태 튜플.`index(찾을_값)`

```
status = ("정상", "정상", "경고", "정상")
```

```
print(status.index("경고"))
```

```
# 없는 값을 찾으면 오류
```

```
# print(status.index("위험"))
```

```
# ValueError: tuple.index(x): x not in tuple
```

▶ 2

왜 쓰나 언제 **처음 이상이 생겼는지**를 찾을 때 씁니다. 시간 순서로 쌓인 상태 기록에서 `index("경고")`가 돌려주는 번호는 곧 "몇 번째 측정에서 처음 경고가 났는가"가 됩니다.

응용 `index`는 오류를 내기 때문에 단독으로 쓰면 프로그램이 멈춥니다. 실전에서는 `in`으로 존재 여부를 먼저 확인하고 나서 `index`를 부르는 두 줄 짝으로 씁니다. 이 패턴은 앞으로도 계속 나옵니다.

```
status = ("정상", "정상", "경고", "정상", "경고")

if "경고" in status:
    first = status.index("경고")
    print("첫 경고 위치:", first, "번째 측정")
    print("그 전까지 정상 구간:", status[:first])
else:
    print("경고 없음 — 전 구간 정상")
```

- ▶ 첫 경고 위치: 2 번째 측정
- ▶ 그 전까지 정상 구간: ('정상', '정상')

자주 하는 실수

index는 **첫 번째 위치**만 알려줍니다. 위 예시에서 경고는 2번과 4번 두 곳에 있지만 결과는 2뿐입니다. 전부 찾으려면 반복문으로 돌면서 직접 모아야 합니다.

왜 count는 0을 주고 index는 오류를 낼까

수업 중 나온 질문입니다. 같은 "찾기"인데 없을 때 반응이 다릅니다. 답은 **두 메서드가 던지는 질문이 다르기** 때문입니다.

메서드	던지는 질문	없을 때	이유
<code>.count(x)</code>	"몇 개 있니?"	0 반환	0개도 정상적인 대답이다. 아예 없는 것도 개수의 일종
<code>.index(x)</code>	"어디 있니?"	ValueError	없는 것의 위치는 대답 자체가 불가능하다

개념

강사님도 "왜 이렇게 설계했냐"는 답을 찾아보진 않았다고 하셨지만, 반환값의 **의미**로 보면 자연스럽습니다. count는 "개수"를 묻는 질문이라 0이라는 답이 성립합니다. 반면 index는 "위치 번호"를 묻는 질문인데, 없는 값에 대해 0을 돌려주면 "0번 자리에 있다"는 **거짓말**이 되어버립니다. 그래서 파이썬은 차라리 오류로 멈춰 세우는 쪽을 택했습니다. 언어 설계에서 애매한 값을 돌려주느니 명확히 실패시키는 편이 낫다는 원칙이 여기에 들어 있습니다.

STEP 2

튜플 리스트와 for 언패킹

오늘의 주력 구조

강사님이 "오늘의 하이라이트"라고 못 박은 부분입니다. 바깥은 리스트, 안은 튜플 — [(이름, 값), (이름, 값), ...] 형태입니다. 바깥이 리스트인 이유는 센서가 계속 추가되기 때문이고, 안이 튜플인 이유는 이름과 값의 짝이 절대 어긋나면 안 되기 때문입니다. **자유로워야 할 곳은 리스트, 지켜야 할 곳은 튜플**이라는 역할 분담이 이 한 줄에 담겨 있습니다.

패턴 [(name, value), ...] 튜플들을 리스트에 모은 구조

정의 리스트의 각 요소가 튜플인 구조. 표(table) 한 줄이 튜플 하나, 표 전체가 리스트에 대응합니다.

```
hour_13 = [
    ("모터온도", 77),
    ("모터진동", 0.2),
    ("모터압력", 91),
]

print(len(hour_13))      # 센서 3개
print(hour_13[0])        # 첫 번째 튜플 전체
print(hour_13[0][0])     # 첫 튜플의 첫 값

▶ 3
▶ ('모터온도', 77)
▶ 모터온도
```

왜 쓰나 센서 이름과 값을 각각 다른 리스트에 넣으면, 한쪽만 정렬하거나 한쪽만 삭제하는 순간 **짝이 통째로 어긋납니다**. 모터온도 자리에 진동값이 붙는 사고가 나는 거죠. 튜플로 묶어두면 어떤 정렬을 해도 짝은 함께 움직입니다.

응용 대괄호를 두 번 쓰면 "몇 번째 센서의 몇 번째 값"으로 정확히 짚을 수 있습니다. 다만 [0][1] 같은 코드는 읽는 사람이 무슨 값인지 알 수 없으므로, 실전에서는 아래 언패킹을 씁니다. 인덱스 두 번은 **구조를 확인할 때만** 쓰는 도구라고 생각하면 됩니다.

설비 적용

설비 점검 리포트가 정확히 이 모양입니다. 한 행이 (설비명, 온도, 진동)이고 전체가 리스트. 나중에 pandas로 넘어가면 이 리스트를 그대로 DataFrame에 넣어 표로 만들 수 있습니다. 지금 이 구조에 익숙해지는 것이 곧 다음 단계 준비입니다.

패턴 for name, value in 리스트 반복하며 자동 언패킹

정의 튜플 리스트를 for로 돌면서 반복 변수를 **여러 개** 쓰면, 매 회차마다 튜플이 자동으로 풀려 각 변수에 들어갑니다.

형태 for 변수1, 변수2 in 튜플_리스트:

```
temps_13 = [
    ("qox_001", 81),
    ("qox_002", 88),
    ("qox_003", 95),
    ("qox_004", 89),
]
warning = 90

for name, temp in temps_13:
    if temp >= warning:
        print("경고", name, "설비 온도 이상")
```

▶ 경고 qox_003 설비 온도 이상

왜 쓰나 인덱스가 필요 없어집니다. row[0], row[1] 대신 name, temp라고 쓰면 **코드가 스스로 설명**합니다. 그리고 센서가 3개든 300개든 이 코드는 한 글자도 바뀌지 않습니다 — 강사님이 말한 "확장성"이 이것입니다.

응용 기준값을 warning = 90처럼 **변수로 빼두는 습관**이 중요합니다. 강사님이 굳이 변수를 쓴 이유를 설명하셨는데, 현장 기준은 계절이나 설비 노후도에 따라 바뀌기 때문입니다. 숫자를 코드 곳곳에 박아두면 기준이 바뀔 때 전부 찾아 고쳐야 하지만, 변수 한 줄만 바꾸면 끝나게 만드는 것이 실무 방식입니다.

```
sensors = [("모터온도", 78), ("회전속도", 1750), ("펌프압력", 95), ("유량", 42)]
LIMIT = 90
```

```
warn_list = []
for name, value in sensors:
    if value > LIMIT:
        warn_list.append(name)

print("점검 대상", len(warn_list), "건:", warn_list)
```

▶ 점검 대상 2 건: ['회전속도', '펌프압력']

개념

변수 이름은 자유입니다. for a, b in이라고 써도 동작은 같습니다. 하지만 for name, temp in이라고 쓰는 순간 코드가 문서가 됩니다. **변수 이름은 미래의 나에게 남기는 주석**입니다.

문법 변수 개수 = 튜플 값 개수 언패킹의 절대 규칙

정의 왼쪽 변수의 개수와 튜플에 담긴 값의 개수가 **정확히 같아야** 합니다. 하나라도 어긋나면 ValueError입니다.

```
tup_list = [
    ("일", "one", 1, "1"),
    ("이", "two", 2, "2"),
]

# 값이 4개 → 변수도 4개
for kor, eng, num, num_str in tup_list:
    print(kor, eng, num, num_str)
```

▶ 일 one 1 1

▶ 이 two 2 2

왜 쓰나 튜플 안의 값이 늘어나면 for문의 변수도 그만큼 늘려야 합니다. 규칙 자체는 단순하지만, 데이터 구조가 바뀌었는데 반복문을 안 고쳐서 나는 오류가 실무에서 상당히 흔합니다.

응용 오류 메시지가 **정확히 몇 개를 기대했고 몇 개가 왔는지** 알려주므로, 메시지만 읽어도 원인을 즉시 알 수 있습니다.

아래 두 메시지를 구분해서 기억해 두면 디버깅이 빨라집니다.

```
# 변수가 더 많을 때
one, two, three, four = (1, 2, 3)
# ValueError: not enough values to unpack (expected 4, got 3)

# 변수가 더 적을 때
one, two = (1, 2, 3)
# ValueError: too many values to unpack (expected 2)
```

자주 하는 실수

언패킹은 튜플 전용이 아닙니다. `one, two, three, four = [11, 22, 33, 44]`처럼 **리스트도 언패킹됩니다**. 수업에서 직접 확인한 부분입니다. 순서가 있는 자료형이면 대체로 가능하다고 이해하면 됩니다.

내장함수 `enumerate()` + 언패킹 번호와 튜플을 동시에 받기

정의 `enumerate`는 (순번, 값) 형태로 돌려줍니다. 값 자체가 튜플이면 **괄호로 한 번 더 묶어** 받아야 합니다.

형태 for 순번, (변수1, 변수2, ...) in `enumerate(튜플_리스트, start=1)`:

```
sensors = [("컨베이어_01", 78, 2.1), ("용접기_02", 92, 5.4)]

for idx, (name, temp, vib) in enumerate(sensors, start=1):
    print(idx, name, temp, vib)
```

- ▶ 1 컨베이어_01 78 2.1
- ▶ 2 용접기_02 92 5.4

왜 쓰나 리포트를 출력할 때 "1. / 2. / 3." 번호를 붙이려면 반드시 필요합니다. 오늘 종합 실습 1이 정확히 이 형태를 요구했습니다. `start=1`을 주면 0이 아니라 1부터 세므로 사람이 읽는 번호와 맞아떨어집니다.

응용 번호를 세려고 `now = 0` 같은 변수를 만들고 `now += 1` 하는 방법도 있습니다. 수업에서도 두 방법을 다 보여주셨는데, `enumerate`가 더 짧고 실수할 여지가 없습니다. 누적 변수는 "몇 번째"가 아니라 "몇 건"을 셀 때 쓰는 게 맞습니다.

자주 하는 실수

괄호를 빼고 `for idx, name, temp, vib in enumerate(sensors):`라고 쓰면 `ValueError: not enough values to unpack (expected 4, got 2)`가 납니다. `enumerate`가 주는 것은 **항상 2개짜리 묶음**이기 때문입니다. 안쪽 튜플은 괄호로 감싸 "이건 하나의 덩어리"라고 알려줘야 합니다.

STEP 3

중첩 튜플과 sorted 정렬

메타데이터 담기 · 정렬 기준 설계하기

센서 하나에 이름과 값만 있는 건 아닙니다. 설치 위치, 허용 범위, 제조사처럼 **부가 정보(메타데이터)**가 따라붙습니다. 이때 튜플 안에 튜플을 한 겹 더 넣습니다. 그리고 이렇게 모은 데이터는 결국 **줄을 세워야** 쓸모가 생깁니다 — 가장 뜨거운 설비부터 보고 싶으니까요.

튜플 중첩 튜플 튜플 안에 튜플 넣기

정의 튜플의 요소로 또 다른 튜플을 넣은 구조. 대괄호를 **두 번** 써서 안쪽 값에 접근합니다.

```
sensor = ("모터온도", 78, (3, 5))

print(sensor[2])      # 안쪽 튜플 전체
print(sensor[2][0])   # 그 안의 첫 값 = 가로좌표
print(sensor[2][1])   # 세로좌표

> (3, 5)
> 3
> 5
```

왜 쓰나 위치 좌표처럼 **두 값이 한 쌍으로만 의미를 갖는** 정보는 따로 두면 안 됩니다. $x=3$, $y=5$ 를 별개 요소로 넣으면 튜플이 ("모터온도", 78, 3, 5)가 되어 3과 5가 각각 무슨 값인지 알 수 없어집니다. 묶어두면 "이 둘은 한 쌍"이라는 사실이 구조에 드러납니다.

응용 허용 범위(하한, 상한)를 함께 담아두면 판정 기준을 데이터가 스스로 들고 다니게 됩니다. 설비마다 기준이 다를 때 코드에 if를 여러 개 쓰지 않아도 되므로 훨씬 깔끔합니다.

```

params = [
    ("Reverb Decay", 72, (0, 100)),
    ("Wet Mix", 45, (0, 100)),
    ("Pre-delay", 28, (0, 200)),
]

for name, value, bound in params:
    min_v, max_v = bound          # 안쪽 튜플을 다시 언패킹
    ratio = value / max_v * 100
    print(f"{name}: {value} (범위 {min_v}~{max_v}, 상한 대비 {ratio:.0f}%)")

```

- ▶ Reverb Decay: 72 (범위 0~100, 상한 대비 72%)
- ▶ Wet Mix: 45 (범위 0~100, 상한 대비 45%)
- ▶ Pre-delay: 28 (범위 0~200, 상한 대비 14%)

개념

for name, value, bound in params: 다음 줄의 min_v, max_v = bound가 **2단 언패킹**입니다. 바깥 튜플은 for가 풀어주고, 안쪽 튜플은 내가 한 줄 더 써서 풉니다. for name, value, (min_v, max_v) in params:처럼 for문에서 한 번에 푸는 것도 가능합니다.

내장함수 sorted() 정렬된 새 리스트 만들기

정의 대상을 정렬해 **새 리스트**로 돌려줍니다. 원본은 건드리지 않습니다.

형태 sorted(대상, reverse=False)

```

temps_13 = [
    (81, "qox_001"),
    (88, "qox_002"),
    (95, "qox_003"),
    (89, "qox_004"),
]

hot = sorted(temps_13, reverse=True)
print(hot)
print("원본:", temps_13)

```

- ▶ [(95, 'qox_003'), (89, 'qox_004'), (88, 'qox_002'), (81, 'qox_001')]
- ▶ 원본: [(81, 'qox_001'), (88, 'qox_002'), (95, 'qox_003'), (89, 'qox_004')]

왜 쓰나 튜플은 불변이라 .sort() 메서드가 없습니다. 그래서 튜플이 든 리스트를 정렬할 때는 sorted를 씁니다.

원본이 보존되므로 **원래 시간 순서를 잃지 않고** 순위표만 따로 뽑을 수 있는 것이 큰 장점입니다.

응용 sorted가 새 리스트를 돌려준다는 말은, **결과를 변수에 담지 않으면 그대로 사라진다는** 뜻이기도 합니다.

강사님이 이 부분을 특히 강조하셨습니다. 상위 N개만 필요하면 정렬 결과에 슬라이싱을 이어 붙이면 한 줄로 끝납니다.

```
temps = [(81, "qox_001"), (88, "qox_002"), (95, "qox_003"), (89, "qox_004")]
```

```
top3 = sorted(temps, reverse=True)[:3] # 정렬 먼저, 자르기 나중
for rank, (temp, name) in enumerate(top3, start=1):
    print(f"{rank}위: {name} - {temp}°C")
```

- ▶ 1위: qox_003 - 95°C
- ▶ 2위: qox_004 - 89°C
- ▶ 3위: qox_002 - 88°C

자주 하는 실수

`sorted(temps)[:3]`과 `sorted(temps[3])`은 **완전히 다릅니다**. 앞은 전체를 정렬한 뒤 상위 3개, 뒤는 앞쪽 3개만 잘라 정렬한 것입니다. 순서를 바꾸면 "상위 3개"가 아니게 됩니다 — 종합 실습 2에서 실제로 나온 실수입니다.

패턴 정렬 기준을 맨 앞에 튜플 순서로 정렬 기준 정하기

정의 튜플 리스트를 정렬하면 **첫 번째 값**을 기준으로 줄을 셉니다. 그래서 정렬하고 싶은 값을 튜플 맨 앞에 배치합니다.

```
# 이름 기준 정렬 (이름이 앞)
a = [("qox_003", 95), ("qox_001", 81)]
print(sorted(a))
```

```
# 온도 기준 정렬 (온도가 앞)
b = [(95, "qox_003"), (81, "qox_001")]
print(sorted(b))
```

- ▶ [('qox_001', 81), ('qox_003', 95)]
- ▶ [(81, 'qox_001'), (95, 'qox_003')]

왜 쓰나 튜플은 앞에서부터 차례로 비교합니다. 첫 값이 같으면 그때 두 번째 값을 봅니다. 이 규칙을 알면 **데이터를 담는 순서를 바꾸는 것만으로** 정렬 기준을 바꿀 수 있습니다. 별도 문법을 배우지 않고도 원하는 순위표가 나옵니다.

응용 첫 값이 같을 때 두 번째 값이 자동으로 tie-breaker가 됩니다. `reverse=True`면 이름도 내림차순으로 갈리므로, 동점자 순서가 사전 역순이 되는 점을 알고 있어야 결과 예측이 맞습니다. 종합 실습 3의 예상 출력이 정확히 이 규칙을 따릅니다.

```
counts = [(4, "TZ_11"), (5, "TZ_13"), (4, "TZ_15"), (1, "TZ_17")]

print(sorted(counts, reverse=True))
# 4회 동점인 TZ_11과 TZ_15 → 이름 내림차순이라 TZ_15가 먼저
▶ [(5, 'TZ_13'), (4, 'TZ_15'), (4, 'TZ_11'), (1, 'TZ_17')]
```

개념

정렬 기준을 따로 지정하는 `key=` 옵션도 있습니다. 강사님이 마우스를 올려 보여주시면서 **"지금은 알아만 두라"**고 하셨는데, `key`를 제대로 쓰려면 함수를 먼저 배워야 하기 때문입니다. 함수를 배우기 전까지는 **정렬 기준을 튜플 맨 앞에 두는 방식**으로 충분히 해결됩니다.

sort() 와 sorted() — 다시 한 번 정리

6일차 리스트에서 배운 `.sort()`와 오늘의 `sorted()`는 이름이 비슷해서 계속 헷갈립니다. **원본을 바꾸느냐, 새로 만드느냐**의 차이 하나만 붙잡으면 됩니다.

	<code>.sort()</code>	<code>sorted()</code>
종류	메서드 (점 찍고 호출)	내장 함수 (괄호에 넣음)
원본	직접 바꿈 (파괴적)	그대로 보존
반환값	None	정렬된 새 리스트
튜플에	사용 불가 (불변이라 없음)	사용 가능 (리스트로 반환)
셋에	사용 불가	사용 가능 (리스트로 반환)
쓸 때	원본 순서가 필요 없을 때	원본을 지켜야 할 때

자주 하는 실수

`lst = lst.sort()`라고 쓰면 `lst`가 **None이 됩니다**. `.sort()`는 정렬만 하고 아무것도 돌려주지 않기 때문입니다. 올바른 사용은 `lst.sort()` 한 줄로 끝내거나(원본 정렬), `new = sorted(lst)`로 받는 것(원본 보존) 둘 중 하나입니다. 이 실수는 오늘 종합 실습 2에서 실제로 발생했고, STEP 9에서 자세히 해부합니다.

STEP 4

셋 — 중복을 지우는 자료형

종류가 궁금할 때 쓰는 그릇

현장 데이터에는 중복이 반드시 섞입니다. 여러 장비가 같은 센서를 읽거나, 통신이 끊겨 재전송되거나, 여러 로그를 합치는 과정에서 같은 기록이 여러 번 들어옵니다. 이때 "몇 건인가"와 "몇 종류인가"는 전혀 다른 질문입니다. 3종류가 10건으로 세어지면 설비 상태를 실제보다 나쁘게 판단하게 됩니다. 셋은 이 문제를 한 줄로 해결합니다.

자료형 set 중복 없는 값의 묶음

정의 중괄호 { }로 감싼 값의 묶음. 중복이 자동으로 제거되고 순서가 없습니다.

형태 {값1, 값2, ...} 또는 set(리스트)

```
# 방법 1 - 중괄호에 값을 직접 나열
unique = {"S01", "S02", "S01", "S03", "S01"}
print(unique)

# 방법 2 - 리스트를 set()으로 감싸기 (실무에서 훨씬 좋음)
logs = ["S01", "S02", "S01", "S03", "S01"]
unique = set(logs)
print(unique)
print(type(unique))

▶ {'S01', 'S02', 'S03'}
▶ {'S01', 'S02', 'S03'}
▶ <class 'set'>
```

왜 쓰나 중복 제거 로직을 직접 짜려면 반복문 + 조건문 + 리스트 검사로 대여섯 줄이 필요합니다. set(logs) 한 줄이면 끝납니다. **가장 빠른 중복 제거 방법**입니다.

응용 set()으로 감싼 뒤 len()을 씌우면 종류 수가 나오고, sorted()를 씌우면 정렬된 목록이 나옵니다. 이 세 줄 조합이 로그 분석의 첫 단계로 거의 항상 등장합니다.

```
logs = ["S01", "S02", "S01", "S03", "S01"]

print("전체 기록:", len(logs), "건")
print("센서 종류:", len(set(logs)), "종")
print("목록:", sorted(set(logs)))
print("중복률:", f"{{(1 - len(set(logs)) / len(logs)) * 100:.0f}}%")
```

```
▶ 전체 기록: 5 건
▶ 센서 종류: 3 종
▶ 목록: ['S01', 'S02', 'S03']
▶ 중복률: 40%
```

자주 하는 실수

{logs}처럼 리스트를 중괄호로 감싸면 오류가 납니다. `TypeError: unhashable type: 'list'` — 셋의 원소는 그 자체로 변하지 않는 값이어야 하는데 리스트는 변할 수 있기 때문입니다. 리스트를 셋으로 바꾸려면 반드시 `set(logs)` 함수를 씁니다.

문법 빈 셋 만들기 {}는 셋이 아니다

정의 빈 중괄호 {}는 셋이 아니라 **딕셔너리**입니다. 빈 셋은 반드시 `set()`으로 만듭니다.

```
empty_list = []
print(type(empty_list))      # <class 'list'>

empty_tuple = ()
print(type(empty_tuple))     # <class 'tuple'>

empty_set = {}
print(type(empty_set))       # <class 'dict'> ← 함정!

real_empty_set = set()
print(type(real_empty_set))  # <class 'set'>
```

```
▶ <class 'list'>
▶ <class 'tuple'>
▶ <class 'dict'>
▶ <class 'set'>
```

왜 쓰나 셋과 딕셔너리가 같은 중괄호를 쓰기 때문에 생긴 문제입니다. 딕셔너리가 파이썬에 먼저 자리를 잡았고, {}는 이미 "빈 딕셔너리"로 굳어져 있었습니다. 그래서 뒤늦게 온 셋이 `set()`이라는 별도 표기를 쓰게 된 것입니다.

응용 경고 센서를 하나씩 모을 때 빈 셋으로 시작해 add로 채워 넣는 것이 전형적인 패턴입니다. 이때 {}로 시작하면 딕셔너리가 되어 add 메서드가 없다는 오류로 이어집니다. **빈 그릇으로 시작할 땐 set() 이라고 외워두면 됩니다.**

```
alerts = set()          # 빈 셋으로 시작
readings = [("S01", 95), ("S02", 78), ("S01", 99), ("S03", 92)]

for name, value in readings:
    if value > 90:
        alerts.add(name)    # 중복은 알아서 무시된다

print("경고 발생 센서:", sorted(alerts))
print("종류 수:", len(alerts))
```

▶ 경고 발생 센서: ['S01', 'S03']
▶ 종류 수: 2

주의

세 개를 나란히 외우면 헛갈리지 않습니다 — 빈 리스트 [], 빈 튜플 (), 빈 셋 set(). 셋만 함수 호출 형태라는 점이 유일한 예외입니다.

문법 셋의 두 가지 성질 무순서 · 무중복

정의 셋은 순서가 없고 중복이 없습니다. 순서가 없으므로 인덱스로 값을 꺼낼 수 없습니다.

```
unique = set(["S01", "S02", "S01", "S03", "S01"])

print(unique)          # 넣은 순서와 다르게 나올 수 있다
print(len(unique))     # 종류 수는 셀 수 있다

# print(unique[0])
# TypeError: 'set' object is not subscriptable
```

▶ {'S01', 'S02', 'S03'}
▶ 3

왜 쓰나 "있다/없다"와 "몇 종류인가"만 따지는 자료형이기 때문입니다. 순서를 버린 대가로 가볍고 빠릅니다 — STEP 5에서 실제 속도 차이를 확인합니다. 순서가 중요하면 리스트, 종류·포함·비교가 중요하면 셋을 고릅니다.

응용 값이 짧을 때는 출력이 정렬된 것처럼 보이는데, 이건 우연입니다. 순서를 보장받고 싶다면 반드시 sorted()를 거쳐야 합니다. 셋을 그대로 출력하고 "왜 순서가 매번 다르지?"라고 당황하지 않으려면 출력 직전에 sorted를 붙이는 습관을 들이는 게 좋습니다.

자주 하는 실수

unique[0]처럼 인덱스로 접근하면 `TypeError: 'set' object is not subscriptable`이 납니다. subscript(첨자)를 지원하지 않는다는 뜻으로, 대괄호 접근 자체가 불가능하다는 의미입니다.

리스트 · 튜플 · 셋 — 무엇을 고를 것인가

자료형이 세 개가 되었으니 이제는 **고르는 기준**이 필요합니다. 강사님이 반복해서 말한 판단 순서를 정리하면 질문 세 개로 압축됩니다. 위에서부터 차례로 답해 내려가면 답이 하나로 좁혀집니다.

질문	예	아니오
① 값이 나중에 바뀌는가?	리스트 또는 셋	튜플 — 짝을 보호해야 하는 데이터
② 순서가 의미를 갖는가?	리스트 또는 튜플	셋 — 순서를 버리고 속도를 얻는다
③ 같은 값이 여러 번 필요한가?	리스트 (횟수를 세야 하므로)	셋 — 종류만 알면 되는 경우

세 질문이 항상 깔끔하게 갈리지는 않습니다. 오늘 종합 실습 3이 좋은 예입니다 — **센서 종류는 셋으로, 경고 횟수는 원본 리스트로** 각각 구해야 합니다. 셋으로 바꾸는 순간 횟수 정보가 사라지기 때문입니다. 하나를 고르는 것이 아니라 **목적별로 나눠 쓰는 것이** 실전에 더 가깝습니다.

두 자료형을 나눠 쓰는 예

```
logs = ["S01", "S02", "S01", "S03", "S01"]
```

```
kinds = set(logs)                                # 종류를 알고 싶을 때 → 셋
print("센서 종류:", len(kinds))
```

```
for k in sorted(kinds):                          # 횟수를 알고 싶을 때 → 원본 리스트
    print(f" {k}: {logs.count(k)}회")
```

```
▶ 센서 종류: 3
▶ S01: 3회
▶ S02: 1회
▶ S03: 1회
```

설비 적용

현장 데이터로 바꿔 말하면 이렇습니다. **"어떤 센서가 문제였나"**는 셋의 질문이고, **"몇 번이나 문제였나"**는 리스트의 질문입니다. 첫 번째 질문은 점검 대상 목록을 뽑을 때 쓰고, 두 번째 질문은 우선순위를 정할 때 씁니다. 두 답이 모두 필요하므로 실무에서는 두 자료형이 나란히 등장합니다.

STEP 5

셋 다루기

add · in · 정렬하면 리스트가 된다

셋은 리스트보다 메서드가 훨씬 적습니다. 순서를 관리하지 않으니 insert도 sort도 필요 없기 때문입니다. 대신 셋이 잘하는 일 — **넣기, 있는지 확인하기, 두 목록 비교하기** — 에 특화된 기능만 남았습니다. 이 STEP에서는 앞의 두 가지를 다룹니다.

메서드 `.add()` 셋에 값 추가

정의 셋에 값을 하나 추가합니다. 이미 있는 값이면 **조용히 무시**합니다. 오류가 나지 않습니다.

형태 셋.add(추가할_값)

```
alerts = {"S01", "S02"}

alerts.add("S03")      # 새 값 → 추가됨
print(alerts)

alerts.add("S01")      # 이미 있는 값 → 무시
print(alerts)

print(alerts.add("S04")) # 반환값은 None

▶ {'S01', 'S02', 'S03'}
▶ {'S01', 'S02', 'S03'}
▶ None
```

왜 쓰나 경고가 발생한 센서만 모으고 싶을 때, **몇 번 올렸는지는 상관없고 어떤 센서가 올렸는지만** 알고 싶은 상황이 있습니다. 리스트의 `append`를 쓰면 같은 센서가 계속 쌓이므로 나중에 중복을 지워야 하지만, 셋의 `add`는 검사 없이 그냥 넣어도 알아서 하나만 남습니다.

응용 리스트로 같은 일을 하려면 `if name not in alerts:` 검사를 매번 붙여야 합니다. 셋을 쓰면 그 조건문이 통째로 사라집니다. 코드 줄이 줄어드는 것보다, **검사를 빼뜨려 생기는 버그가 원천 차단**되는 것이 더 큰 이득입니다.

```
# 리스트로 할 때
alerts_list = []
for name in ["S01", "S02", "S01", "S03"]:
    if name not in alerts_list:      # 이 줄을 빠뜨리면 중복이 쌓인다
        alerts_list.append(name)

# 셋으로 할 때
alerts_set = set()
for name in ["S01", "S02", "S01", "S03"]:
    alerts_set.add(name)            # 검사 자체가 불필요

print(alerts_list, sorted(alerts_set))
```

```
▶ ['S01', 'S02', 'S03'] ['S01', 'S02', 'S03']
```

주의

`.add()`는 `None`을 돌려줍니다. 즉 `alerts = alerts.add("S05")`라고 쓰면 셋이 통째로 사라집니다.

`.sort()`와 같은 함정입니다 — **원본을 직접 바꾸는 메서드는 대체로 `None`을 돌려준다고** 묶어서 기억해 두세요.

연산자 `in` 포함 여부 확인

정의 값이 셋(또는 리스트·튜플·문자열) 안에 있으면 `True`, 없으면 `False`를 돌려줍니다.

형태 값 `in` 대상

```
alerts = {"S01", "S02", "S03"}

print("S02" in alerts)
print("S09" in alerts)

# 실전에서는 이렇게 조건문과 함께 쓴다
if "S02" in alerts:
    print("S02 정비 알림")
```

```
▶ True
▶ False
▶ S02 정비 알림
```

왜 쓰나 `True/False`를 그대로 출력하는 일은 거의 없습니다. **조건문의 조건으로 넣어** "경고 목록에 있을 때만 알림을 보낸다", "이미 처리한 센서면 건너뛰다" 같은 판단을 만드는 것이 실제 용도입니다.

응용 셋의 `in`이 **압도적으로 빠릅니다**. 리스트는 처음부터 하나씩 비교하지만, 셋은 값이 있어야 할 자리를 미리 계산해 두어 곧장 찾아갑니다. 아래는 값 5만 개에서 200번 검색한 실측치입니다.

```
import timeit
L = [f"S{i:05d}" for i in range(50000)]
S = set(L)
target = "S49999"      # 최악의 경우 = 맨 끝

print("리스트:", timeit.timeit(lambda: target in L, number=200))
print("셋 :", timeit.timeit(lambda: target in S, number=200))
```

▶ 리스트: 0.1231초

▶ 셋 : 0.000026초 → 약 4,700배 차이

개념

강사님 표현을 빌리면 "사람마다 자리를 정해 하나씩 확인하는 것"과 "한 주머니에 넣어두고 바로 꺼내는 것"의 차이입니다. 지금 다루는 데이터는 길어야 수십 개라 체감이 안 되지만, 센서 수천 개의 로그를 다룰 때는 이 차이가 그대로 실행 시간이 됩니다.

내장함수 sorted(셋) 셋을 정렬하면 리스트가 된다

정의 셋을 sorted()에 넣으면 정렬된 **리스트**가 나옵니다. 셋이 아니라 리스트입니다.

```
alerts = {"S03", "S01", "S02"}

result = sorted(alerts)
print(result)
print(type(result))
```

▶ ['S01', 'S02', 'S03']
▶ <class 'list'>

왜 쓰나 수업 중 나온 질문이 그대로 답입니다 — "정렬한다는 것은 순서가 필연적으로 따라오는 개념"입니다. 셋은 순서라는 개념 자체가 없으므로 정렬된 셋이라는 것은 존재할 수 없습니다. 그래서 파이썬은 순서를 담을 수 있는 리스트로 바꿔 돌려줍니다.

응용 셋으로 중복을 지우고 → 정렬해 보기 좋게 만드는 두 단계는 거의 항상 붙어 다닙니다. 그래서 sorted(set(logs)) 한 줄이 관용구처럼 쓰입니다. 순서를 되찾고 싶으면 셋 → 리스트로 돌아오면 됩니다.

```
logs = ["S03", "S01", "S02", "S01", "S03"]

clean = sorted(set(logs))          # 중복 제거 + 정렬
print(clean)
print("종류 수:", len(clean))
print("첫 번째:", clean[0])        # 리스트가 되었으니 인덱스 사용 가능
```

▶ ['S01', 'S02', 'S03']

▶ 종류 수: 3

▶ 첫 번째: S01

자주 하는 실수

수업 코드에 `sorted = sorted(alerts)`라고 쓴 부분이 있었습니다. 동작은 하지만 **내장 함수 이름을 변수로 덮어써버립니다**. 그 뒤로 `sorted(...)`를 호출하면 `TypeError: 'list' object is not callable`이 납니다. 내장 함수 이름(`sorted`, `list`, `set`, `sum`, `len`, `type`, `str`, `id`)은 변수 이름으로 쓰지 않습니다.

셋이 가벼운 이유

강사님이 CS(컴퓨터 사이언스) 이야기를 잠깐 곁들이며 설명한 부분입니다. 지금 당장 다 이해할 필요는 없지만, **"왜 빠르냐"**를 한 줄로 답할 수 있으면 면접에서도 유용합니다.

기준	리스트	셋
저장하는 것	값 + 순서 정보	값만 (순서 관리 안 함)
메모리	상대적으로 많이 차지	적게 차지
길이	중복 포함 전체	중복 제거 후 → 더 짧음
in 검색	앞에서부터 하나씩 비교	자리를 계산해 바로 접근
내장 메서드 수	많음	리스트보다 적고 튜플보다 많음

설비 적용

센서가 열 개뿐이라면 리스트든 셋이든 차이를 못 느낍니다. 하지만 실제 설비 로그는 하루에도 수만 건이 쌓입니다. "이 센서가 오늘 경고 목록에 있었나?"를 수만 번 물어야 한다면, 리스트로 하면 몇 초가 걸릴 일이 셋으로는 순식간에 끝납니다. **자료형 선택이 곧 성능**이라는 감각을 지금부터 들여두면 좋습니다.

STEP 6

집합 연산 3종 세트

전체는 합집합, 공통은 교집합, 차이는 차집합

중학교 수학의 벤 다이어그램이 그대로 코드가 됩니다. 동그라미 두 개가 겹친 그림 하나에 세 연산이 전부 들어 있습니다. 강사님이 강조한 원칙은 "두 목록을 받으면 습관적으로 세 연산을 다 구해보라"는 것입니다. 그러면 전체 규모, 공통점, 각자의 고유 영역이 한눈에 정리됩니다.

예제로 쓸 데이터는 두 시간대의 이상 센서 목록입니다.

공통 데이터

```
hour_14 = {"WQR_01", "WQR_06", "WQR_07", "WQR_02"}
hour_15 = {"WQR_01", "WQR_07", "WQR_03", "WQR_09", "WQR_11"}
```

매서드 .union() / | 합집합 — 전체 종류 파악

정의 두 셋을 합쳐 **중복 없는 전체 목록**을 만듭니다. 파이프 기호 |로도 같은 일을 합니다.

형태 A.union(B) 또는 A | B

```
print(sorted(hour_14.union(hour_15)))
print(sorted(hour_15.union(hour_14))) # 순서 바뀌도 같은 결과
print(sorted(hour_14 | hour_15))      # 연산자 표기

print(sorted(hour_14))                # 원본은 변화 없음

▶ ['WQR_01', 'WQR_02', 'WQR_03', 'WQR_06', 'WQR_07', 'WQR_09', 'WQR_11']
▶ ['WQR_01', 'WQR_02', 'WQR_03', 'WQR_06', 'WQR_07', 'WQR_09', 'WQR_11']
▶ ['WQR_01', 'WQR_02', 'WQR_03', 'WQR_06', 'WQR_07', 'WQR_09', 'WQR_11']
▶ ['WQR_01', 'WQR_02', 'WQR_06', 'WQR_07']
```

왜 쓰나 "이 공장 전체에 센서가 몇 종류인가"를 알 때 씁니다. A라인과 B라인의 센서 목록을 합치면 공통 센서가 두 번 세어지지 않고 정확한 전체 종류가 나옵니다. 라인이 여럿이면 차례로 union해 가면 됩니다.

응용 union은 원본을 바꾸지 않고 새 셋을 돌려줍니다. 그래서 결과를 변수에 담지 않으면 사라집니다. 반대로 원본에 바로 합치고 싶으면 A |= B를 쓰면 되는데, 이건 리스트의 +=와 같은 발상입니다.

```
line_a = {"S01", "S02"}
line_b = {"S02", "S03"}
line_c = {"S04"}
```

```
total = line_a.union(line_b).union(line_c) # 이어 붙이기 가능
print("전체 센서", len(total), "종:", sorted(total))
print("원본 line_a:", sorted(line_a))      # 그대로
```

```
▶ 전체 센서 4 종: ['S01', 'S02', 'S03', 'S04']
▶ 원본 line_a: ['S01', 'S02']
```

개념

A.union(B)와 B.union(A)는 **결과가 같습니다**. 합집합은 순서를 타지 않습니다. 뒤에 나올 차집합과 결정적으로 다른 점입니다.

메서드 .intersection() / & 교집합 — 공통점 찾기

정의 두 셋에 **공통으로 들어 있는 값만** 남깁니다. 앰퍼샌드 &로도 같습니다.

형태 A.intersection(B) 또는 A & B

```
print(sorted(hour_14.intersection(hour_15)))
print(sorted(hour_15.intersection(hour_14))) # 순서 무관
print(sorted(hour_14 & hour_15))
```

```
▶ ['WQR_01', 'WQR_07']
▶ ['WQR_01', 'WQR_07']
▶ ['WQR_01', 'WQR_07']
```

왜 쓰나 "두 조건을 동시에 만족하는 것"을 찾는 연산입니다. 두 라인이 공유하는 부품을 파악할 때, 그리고 이를 연속 경고가 뜬 센서를 추적할 때 강력합니다. 합집합이 가장 넓다면 교집합은 가장 좁습니다.

응용 교집합의 크기 자체가 지표가 됩니다. 어제와 오늘의 이상 센서 교집합이 크면 문제가 해결되지 않고 계속되고 있다는 뜻이고, 작으면 이상이 매번 다른 곳에서 산발적으로 뜬다는 뜻입니다. 같은 데이터라도 **해석이 완전히 달라집니다**.

```
yesterday = {"S01", "S02", "S03"}
today = {"S02", "S03", "S05"}

same = today & yesterday
rate = len(same) / len(today) * 100
print("지속 이상:", sorted(same))
print(f"오늘 이상 중 {rate:.0f}%가 어제부터 계속됨 — 미해결 비율")
```

▶ 지속 이상: ['S02', 'S03']
 ▶ 오늘 이상 중 67%가 어제부터 계속됨 — 미해결 비율

메서드 `.difference()` / - 차집합 — 변화 추적

정의 앞 셋에는 있고 뒤 셋에는 없는 값만 남깁니다. 빼기 기호 -로도 같습니다.

형태 `A.difference(B)` 또는 `A - B`

```
print(sorted(hour_14.difference(hour_15)))
print(sorted(hour_15.difference(hour_14)))
```

```
print(sorted(hour_14 - hour_15))
print(sorted(hour_15 - hour_14))
```

▶ ['WQR_02', 'WQR_06']
 ▶ ['WQR_03', 'WQR_09', 'WQR_11']
 ▶ ['WQR_02', 'WQR_06']
 ▶ ['WQR_03', 'WQR_09', 'WQR_11']

왜 쓰나 두 시점을 비교해 변화를 잡아내는 연산입니다. 오늘 경고에서 어제 경고를 빼면 **오늘 새로 생긴 이상**이 나오고, 방향을 뒤집으면 **오늘 해소된 이상**이 나옵니다. 같은 두 데이터로 서로 다른 두 가지 사실을 뽑아낼 수 있습니다.

응용 교집합·양방향 차집합 세 줄이면 두 시점의 관계가 완전히 분석됩니다. 신규 / 지속 / 해소 세 갈래로 나누는 이 패턴은 실무 리포트에서 그대로 쓰입니다.

```
yesterday = {"S01", "S02", "S03"}
today = {"S02", "S03", "S05"}

print("🆕 신규 이상:", sorted(today - yesterday))
print("☑️ 지속 이상:", sorted(today & yesterday))
print("☑️ 해소된 이상:", sorted(yesterday - today))
```

▶ 🆕 신규 이상: ['S05']
 ▶ ☑️ 지속 이상: ['S02', 'S03']
 ▶ ☑️ 해소된 이상: ['S01']

자주 하는 실수

차집합은 순서에 따라 결과가 완전히 다릅니다. $A - B$ 와 $B - A$ 는 다른 값입니다. 합집합·교집합이 순서를 타지 않는 것과 대조적이라 특히 헛갈립니다. "빼기니까 당연히 방향이 있다"고 산수에 붙여 기억하면 실수가 줄어듭니다.

세 연산 한 장 정리

연산	메서드	연산자	의미	순서 영향	대표 용도
합집합	<code>.union()</code>		합친 전체	없음	전체 종류 파악
교집합	<code>.intersection()</code>	&	공통인 것	없음	공통점 · 지속 이상
차집합	<code>.difference()</code>	-	한쪽에만	있음	변화 추적 · 신규/해소

연결

강사님이 마지막에 덧붙인 말이 중요합니다 — **이 집합 연산은 나중에 pandas로 두 개의 표를 비교할 때 그대로 쓰입니다.** 지금은 센서 ID 몇 개를 비교하고 있지만, 같은 개념이 "어제 데이터에는 있는데 오늘 데이터에는 빠진 행 찾기" 같은 작업으로 확장됩니다. 벤 다이어그램 세 칸을 지금 확실히 잡아두면 그때 새로 배울 것이 거의 없습니다.

주의

메서드와 연산자 중 무엇을 쓸지는 취향입니다. 강사님도 "사람마다 다르다"고 하셨습니다. 다만 한 가지 실용적인 기준은 있습니다 — 연산자 |, &, -는 **양쪽이 반드시 셋이어야** 하지만, 메서드 `.union(...)`은 인자로 **리스트를 줘도 알아서 처리**합니다. 데이터 타입이 확실하지 않을 때는 메서드 쪽이 안전합니다.

STEP 7

교안 실습 1~6 복기

내가 제출한 코드 점검

교안에 딸린 실습 여섯 개를 Practice/06_01_example.py에 풀어 제출했습니다. 대체로 잘 풀었고 아직 배우지 않은 문법까지 찾아 쓴 부분도 있습니다. 여기서는 **의도와 다르게 동작한 두 곳**과 **칭찬할 만한 한 곳**을 짚습니다. 채점이 목적이 아니라, 같은 실수를 종합 실습에서 반복하지 않기 위한 점검입니다.

실습 1 · 3 — 정확하게 풀었습니다

실습 1의 인덱싱·언패킹, 실습 3의 **2단 언패킹**은 정확합니다. 특히 실습 3에서 `for name, presence, bound in val:` 다음 줄에 `min_v, max_v = bound`를 쓴 부분은 중첩 튜플의 정석입니다.

제출 코드 — 실습 3

```
val = [
    ("Reverb Decay", 72, (0, 100)),
    ("Wet Mix", 45, (0, 100)),
    ("Pre-delay", 28, (0, 200)),
]

for name, presence, bound in val:
    min_v, max_v = bound          # ← 2단 언패킹, 정확함
    print(name, "최소 최대:", min_v, max_v)
```

- ▶ Reverb Decay 최소 최대: 0 100
- ▶ Wet Mix 최소 최대: 0 100
- ▶ Pre-delay 최소 최대: 0 200

실습 2 — 반복문의 의미가 사라졌습니다

교안 실습 2의 목표는 "(이름, 값) 튜플의 리스트를 **for**로 순회"하는 것이었습니다. 그런데 제출 코드는 튜플 **하나**를 언패킹한 뒤, 꺼낸 값을 다시 1칸짜리 튜플로 감싸 반복문에 넣었습니다.

제출 코드 — 실습 2

```
effector = ("strymon", "Bigsky MX", 36, 127,)
brand, name, decay, parameter = effector

for parameter in (parameter,):      # ← 1회만 반복됨
    if parameter >= 90:
        print("parameter 값 큼", parameter)
```

- ▶ parameter 값 큼 127
- ▶
- ▶ (반복 횟수: 1회)

자주 하는 실수

문제가 두 가지 겹쳐 있습니다. ① **반복 대상이 값 하나뿐**이라 for문을 써도 한 번만 돕니다. 반복문의 목적인 "센서가 몇 개든 같은 코드"가 성립하지 않습니다. ② 반복 변수 이름이 기존 변수 `parameter`와 같아서, 반복이 끝난 뒤 원래 값이 덮어써집니다. 지금은 값이 같아 티가 안 나지만 데이터가 여러 개가 되는 순간 추적이 어려워집니다.

의도대로 고치면 이렇게 됩니다. **여러 개를 튜플 리스트로 담고, for가 그걸 순회**하게 만드는 것이 핵심입니다.

고친 코드

```
effectors = [
    ("Bigsky MX", 36, 127),
    ("Timeline", 48, 84),
    ("Volante", 61, 95),
]
LIMIT = 90

for name, decay, parameter in effectors:
    if parameter >= LIMIT:
        print(f"{name}: parameter {parameter} - 기준 {LIMIT} 초과")
```

- ▶ Bigsky MX: parameter 127 - 기준 90 초과
- ▶ Volante: parameter 95 - 기준 90 초과

실습 4 — 라벨과 출력값이 어긋났습니다

교안 실습 4의 요구는 "**정렬 출력 + 종류 수 확인**" 두 가지였습니다. 제출 코드는 정렬 목록만 출력하면서 라벨은 "종류 개수"라고 붙였습니다. `len()`이 빠진 것입니다.

제출 코드 — 실습 4

```
ids = [f"WOR_{random.randint(1,10):02d}" for _ in range(10)]
unique = sorted(set(ids))
print("종류 개수: ", unique)          # ← 라벨은 개수, 출력은 목록
```

▶ 종류 개수: ['WOR_01', 'WOR_03', 'WOR_07', 'WOR_09']

고친 코드

```
ids = ["WOR_01", "WOR_03", "WOR_01", "WOR_07", "WOR_03"]

unique = set(ids)                # 셋 상태로 먼저 잡고
print("종류 개수:", len(unique))  # 개수는 len으로
print("종류 목록:", sorted(unique))  # 목록은 sorted로
```

▶ 종류 개수: 3

▶ 종류 목록: ['WOR_01', 'WOR_03', 'WOR_07']

주의

사소해 보이지만 실무에서 가장 위험한 종류의 버그입니다. 코드는 오류 없이 돌고, 출력도 그럴듯합니다. 리포트를 읽는 사람만 잘못된 정보를 받습니다. 변수 이름과 라벨 문구를 실제 출력값과 맞추는 습관이 필요합니다. 덧붙이면 `unique = sorted(set(ids))`로 받는 순간 그 변수는 **셋이 아니라 리스트**가 되므로, 이름이 `unique`인 것도 다소 오해를 부릅니다.

실습 5 · 6 — 진도 밖 문법을 찾아 쓴 부분

실습 5·6에서 `{f"WOR_{random.randint(1,10):02d}" for _ in range(7)}` 형태를 썼습니다. 이건 **셋 컴프리헨션(set comprehension)**이라는 문법으로, 아직 수업에서 다루지 않은 범위입니다. 스스로 찾아 적용한 것은 좋은 태도입니다. 다만 학습 단계에서 한 가지 짚어둘 점이 있습니다.

개념

`random`을 쓰면 **실행할 때마다 결과가 달라져 교안의 예상 출력과 대조할 수 없습니다**. 문법이 맞는지는 확인되지만, 로직이 맞는지는 확인이 어렵습니다. 학습 단계에서는 교안과 **같은 고정 데이터**로 풀어 예상 출력과 한 글자씩 맞춰보고, 그게 통과한 뒤에 랜덤 데이터로 바꿔 테스트하는 순서가 효율적입니다. 참고로 컴프리헨션 문법은 진도상 반복문·조건문을 충분히 익힌 뒤 배우게 되며, 그때 `[...]`(리스트) · `{...}`(셋) · `{k: v ...}`(딕셔너리) 세 형태를 한 번에 정리하게 됩니다.

추가로 `import random`이 파일 중간에 있습니다. 파이썬 관례(PEP 8)상 **import**는 **파일 맨 위에 모아 두는 것**이 표준입니다. 지금은 문제가 없지만 파일이 길어지면 "이 모듈을 쓰는지" 확인하기 어려워집니다.

실습 6개 자가 점검표

다시 풀어볼 때 이 표의 오른쪽 칸만 확인하면 됩니다. 문법이 아니라 **의도대로 동작했는지**를 보는 관점입니다.

실습	핵심 확인 사항	결과
1. 튜플 묶고 꺼내기	인덱싱과 언패킹 두 방법을 모두 썼는가	통과
2. 튜플 리스트 반복	반복 대상이 튜플 여러 개인가 (1개면 for가 무의미)	재작성 필요
3. 중첩 튜플	안쪽 튜플을 2단으로 풀었는가	통과
4. 셋 중복 제거	종류 "수"와 종류 "목록"을 각각 출력했는가	len 누락
5. 두 라인 비교	합집합·교집합·양방향 차집합 4개를 다 구했는가	통과
6. 두 시점 추적	신규(차집합)와 지속(교집합)의 방향이 맞는가	통과

개념

실습 5·6은 방향까지 정확했습니다. `today.difference(yesterday)`가 신규, `today.intersection(yesterday)`가 지속 — 이 두 줄을 헷갈리지 않은 것은 집합 연산의 의미를 이해했다는 뜻입니다. 반대로 `yesterday.difference(today)`라고 썼다면 **해소된 이상**이 나왔을 텐데, 이것도 유용한 지표이므로 STEP 6의 3분할 코드처럼 세 줄을 나란히 써두는 것을 권합니다.

STEP 8

종합 실습 1 — 설비 종합 모니터링 리포트

튜플 리스트 총동원

오후 마지막 시간에 진행한 종합 실습입니다. 가이드에 적힌 문장이 이 실습의 성격을 잘 말해줍니다 — "대부분은 몰라서 못 푸는 게 아니라 안 떠올라서 못 푸는 것". 문법을 새로 배우는 시간이 아니라, 배운 것을 꺼내 쓰는 훈련입니다.

판정	조건	해당 설비
위험 ☒	온도 > 90 or 진동 > 5.0	용접기_02(92) · 건조로_04(101, 6.8)
주의 ☑	온도 >= 80 or 진동 >= 3.0	절단기_03 · 도장부스_06 · 성형기_07
정상 ☑	그 외	컨베이어_01 · 냉각탑_05

주의

경계값이 이 문제의 함정입니다. **성형기_07은 온도가 정확히 90입니다.** > 90은 거짓이므로 위험이 아니고, >= 80 이 참이므로 주의입니다. 부등호에 등호가 붙었는지 아닌지를 한 글자 단위로 봐야 합니다. 그리고 elif 순서도 중요합니다 — 주의를 먼저 검사하면 위험 설비까지 전부 주의로 빠집니다. **좁은 조건부터 넓은 조건 순으로** 배치하는 것이 원칙입니다.

제출한 코드 — 잘한 부분

출력 형식이 예상 출력과 거의 완전히 일치합니다. enumerate와 괄호 언패킹을 정확히 조합했고, f-string 안에서 계산과 서식을 함께 처리한 것도 깔끔합니다. 누적 변수 다섯 개를 미리 선언해 둔 구조도 정석입니다.

제출 코드 — 핵심부

```
for idx, (name, temp, vib) in enumerate(sensors, start=1):
    if temp > 90 or vib > 5.0:
        print(f"{idx}. {name} | 온도 {temp}°C | 진동 {vib}mm/s | 위험 ☒")
        num_danger += 1
        danger_list.append(name)

    if peak_temp < temp:           # ← 이 두 줄의 위치가 문제
        peak_temp = temp
        peak_name = name

    elif temp >= 80 or vib >= 3.0:
        ...
    total_temp += temp
```

버그 ① — 최고 온도를 위험 설비 중에서만 찾습니다

peak_temp 갱신 코드가 **위험 판정 블록 안에** 들어가 있습니다. 이번 데이터에서는 최고 온도(101°C)가 마침 위험 설비라 정답이 나왔지만, 이건 **데이터가 맞춰준 우연**입니다.

반례 검증

```
# 위험 설비가 한 대도 없는 데이터로 바꿔보면
sensors = [("A", 88, 2.0), ("B", 85, 1.0)]

# 제출 코드 방식의 결과
# 최고 온도 설비: (0°C) ← 이름이 빈 문자열, 온도가 0

▶ 최고 온도 설비: (0°C)
```

요구사항 5번은 **"전체 설비 중"** 온도가 가장 높은 설비였습니다. 판정과 최댓값 찾기는 서로 독립적인 작업이므로, 갱신 코드는 if/elif 바깥 — 즉 total_temp += temp와 같은 높이에 있어야 합니다.

고친 코드

```
for idx, (name, temp, vib) in enumerate(sensors, start=1):
    if temp > 90 or vib > 5.0:
        ...
    elif temp >= 80 or vib >= 3.0:
        ...
    else:
        ...

    total_temp += temp
    if peak_temp < temp:          # ← 판정과 무관하게 모든 설비를 비교
        peak_temp = temp
        peak_name = name
```

개념

peak_temp = 0으로 초기화하는 것도 온도가 항상 양수라는 전제에 기대고 있습니다. 영하 데이터가 들어오면 최고값을 못 찾습니다. 더 안전한 방법은 **첫 번째 데이터로 초기화**하는 것입니다 — peak_temp = sensors[0][1]. 이 습관은 다음 실습에서 결정적인 차이를 만듭니다.

버그 ② — 위험 설비 목록을 정렬하지 않았습니다

요구사항 6번은 **"모아서 정렬해 리스트로 출력"**이었습니다. .append()는 했지만 .sort()가 빠져서 출력이 예상과 다릅니다.

	출력	원인
제출 코드	<code>['용접기_02', '건조로_04']</code>	append 순서 = 데이터 등장 순서
예상 출력	<code>['건조로_04', '용접기_02']</code>	가나다순 정렬 ($\neg < \circ$)

고친 코드

```
danger_list.sort()                # 원본 정렬
print(f"위험 설비 목록: {danger_list}")

# 또는 출력할 때만 정렬 (원본 순서 보존)
print(f"위험 설비 목록: {sorted(danger_list)}")
```

▶ 위험 설비 목록: `['건조로_04', '용접기_02']`

도전 과제 — 아직 안 한 부분

가이드의 도전 과제는 **위험 설비가 한 대라도 있으면 경고 문구를 붙이는** 것이었습니다. 누적 변수 `num_danger`를 이미 세어 두었으므로 세 줄이면 끝납니다.

도전 과제 추가

```
if num_danger > 0:
    print("⚠ 즉시 점검 요망")
else:
    print("✅ 전 설비 안정")
print("=" * 50)
```

▶ ⚠ 즉시 점검 요망

▶ =====

응용

`if num_danger > 0:` 대신 `if danger_list:`라고 써도 똑같이 동작합니다. 파이썬에서 **빈 리스트는 거짓, 값이 든 리스트는 참**으로 취급되기 때문입니다. `len(danger_list) > 0`보다 짧고 파이썬다운 표현이라 실무 코드에서 자주 씁니다. 같은 원리로 빈 문자열 `""`, 숫자 `0`, 빈 셋 `set()`도 모두 거짓입니다.

STEP 9

종합 실습 2 — 실시간 측정값 입력 시스템

while + 통계 · 버그 5종 해부

작업자가 몇 번 입력할지 모르는 상황이라 while을 씁니다. 8일차에 배운 "횟수를 알면 for, 모르면 while"이 그대로 적용됩니다. 제출 코드는 전체 뼈대는 맞게 잡았지만, 실행해 보면 예상 출력과 상당히 다릅니다. 하나씩 해부해 보겠습니다.

항목	예상 출력	제출 코드 실제 출력
총 입력 개수	3개	2 ← 120이 빠짐
최댓값 / 최솟값	120.0 / 60.0	85 / 0 ← 둘 다 틀림
평균값	88.33	72.5 ← 값·자릿수 모두 다름
임계값 초과 개수	1개	1 ← 맞음
상위 3개 값	[120.0, 85.0, 60.0]	None ← 값이 아예 없음

입력값 85 / 120 / 60 / q로 실행한 결과입니다. 다섯 항목 중 네 개가 틀렸는데, 오류 메시지는 하나도 뜨지 않았습니다. 이런 종류의 버그가 가장 위험합니다.

버그 ① — 임계값 초과 값이 리스트에서 통째로 빠집니다

제출 코드

```
while True:
    val = input("측정값: ")

    if val == "q":
        break
    elif int(val) > 100:                # ← 초과값은 여기서 끝
        warn_count += 1
        print(f"경고☹ - LIMIT 초과 회수: {warn_count}")
    else:                             # ← append는 else에만 있음
        lst.append(int(val))
        total_val += int(val)
```

자주 하는 실수

if / elif / else는 셋 중 하나만 실행됩니다. 120을 입력하면 elif 가지로 가서 경고만 출력하고 끝나므로, lst.append()와 total_val +=가 실행되지 않습니다. 결과적으로 가장 중요한 이상값이 통계에서 빠집니다. 요구사항은 "값은 리스트에 모으기"와 "초과하면 경고"를 둘 다 하라는 것이었으므로, 이 둘은 배타적 분기가 아니라 순차 실행이어야 합니다.

고친 코드

```
while True:
    raw = input("측정값: ")
    if raw == "q":
        break

    val = float(raw)          # 변환은 한 번만, 변수에 담아 재사용
    values.append(val)        # 초과값도 "측정값"이므로 무조건 담는다

    if val > LIMIT:           # 담은 뒤에 별도로 경고를 판단
        over_count += 1
        print(f" □ 임계값({LIMIT}) 초과! 현재까지 초과 {over_count}회")
```

버그 ② — 최솟값 초기화가 0입니다

가장 치명적인 버그입니다. min_val = 0으로 시작하면, 모든 측정값이 양수인 한 min_val > lst[i] 조건이 단 한 번도 참이 되지 않습니다. 그래서 최솟값은 영원히 0으로 출력됩니다.

제출 코드

```
max_val = 0
min_val = 0          # ← 여기가 문제
for i in range(len(lst)):
    if max_val < lst[i]:
        max_val = lst[i]

    if min_val > lst[i]:    # 0보다 작은 값이 없으니 절대 실행 안 됨
        min_val = lst[i]
```

▶ 최댓값: 85 / 최솟값: 0

해법은 첫 번째 값으로 초기화하는 것입니다. "지금까지 본 것 중 가장 큰 값"이라는 의미를 생각하면, 아무것도 안 봤을 때의 초기값으로 0을 쓰는 것 자체가 근거가 없습니다. 첫 값을 넣어두고 두 번째부터 비교하는 것이 논리적으로 자연스럽습니다.

고친 코드

```
max_val = values[0]      # 첫 값으로 시작
min_val = values[0]

for v in values:         # range(len())보다 직접 순회가 읽기 좋다
    if v > max_val:
        max_val = v
    if v < min_val:
        min_val = v
```

▶ 최댓값: 120.0 / 최솟값: 60.0

개념

`for i in range(len(lst))`로 인덱스를 만들어 `lst[i]`로 꺼내는 방식도 동작하지만, 파이썬에서는 `for v in values:`처럼 값을 직접 순회하는 쪽을 선호합니다. 인덱스가 실제로 필요할 때만 `enumerate`를 쓰는 것이 관례입니다. 인덱스를 거치면 `lst[i]`를 매번 타이핑해야 하고, 오프바이원(off-by-one) 실수가 끼어들 여지도 생깁니다.

버그 ③ — 정렬 순서가 거꾸로, 그리고 None

마지막 줄에 두 가지 오류가 한꺼번에 들어 있습니다.

제출 코드

```
print(f"상위 3개 값: {lst[:3].sort(reverse=True)}")
```

▶ 상위 3개 값: None

문제	무슨 일이 일어나나	근거
① 자르기 → 정렬	정렬 안 된 앞 3개를 잘라서 정렬한 것 = 상위 3개가 아님	슬라이싱이 먼저 실행됨
② .sort() 반환값	None이 출력됨 — 정렬 결과는 어디에도 남지 않음	.sort()는 None을 반환

고친 코드

```
values.sort(reverse=True)      # ① 전체를 먼저 정렬
top3 = values[:3]              # ② 그 다음 앞에서 3개
print(f"상위 3개 값: {top3}")

# 원본 순서를 지키고 싶다면 한 줄로
print(f"상위 3개 값: {sorted(values, reverse=True)[:3]}")
```

▶ 상위 3개 값: [120.0, 85.0, 60.0]

버그 ④·⑤ — 상수를 정의해 놓고 쓰지 않았습니다

파일 맨 위에 `LIMIT = 100`과 `quit_str = "q"`를 선언해 두고, 정작 코드에서는 `int(val) > 100`과 `val == "q"`처럼 값을 직접 박아 썼습니다. 지금은 동작하지만 **기준이 바뀌는 순간 버그가 됩니다** — `LIMIT`만 90으로 고치면 경고는 여전히 100 기준으로 뜨고, 화면에는 "임계값(90) 초과"라고 나오게 됩니다.

	제출 코드	고칠 것
임계값	<code>elif int(val) > 100:</code>	<code>if val > LIMIT:</code>
종료 문자	<code>if val == "q":</code>	<code>if raw == quit_str:</code>
형변환	<code>int(val)</code> 을 4번 반복 호출	<code>val = float(raw)</code> 한 번만
평균 자릿수	<code>:.1f</code> → 72.5 (요구는 둘째 자리)	<code>round(avg, 2)</code> → 88.33

개념

`int()`와 `float()` 선택도 짚고 갑니다. 가이드는 `float()`를 지정했는데, 온도·진동 같은 **측정값은 소수점을 가질 수 있기 때문**입니다. `int("85.5")`는 `ValueError`가 납니다 — 정수 형태의 문자열만 받기 때문입니다. 측정 데이터를 다룰 때는 `float()`가 기본이라고 생각하는 편이 안전합니다.

전체 고친 코드

완성본 — 예상 출력과 완전 일치

```
LIMIT = 100
QUIT = "q"
values = []
over_count = 0

while True:
    raw = input("측정값: ")
    if raw == "":
        # 도전) 빈 입력은 무시하고 다시 받기
```

```

        continue
    if raw == QUIT:                # 형변환 전에 문자열로 먼저 비교
        break

    val = float(raw)
    values.append(val)
    if val > LIMIT:
        over_count += 1
        print(f"  ☐ 임계값({LIMIT}) 초과! 현재까지 초과 {over_count}회")

print("-" * 40)
if len(values) == 0:
    print("입력된 측정값이 없습니다.")
else:
    total = 0
    max_val = values[0]
    min_val = values[0]
    for v in values:
        total += v
        if v > max_val: max_val = v
        if v < min_val: min_val = v
    avg = total / len(values)

    over_avg = 0                # 평균을 다 구한 뒤에야 셀 수 있다
    for v in values:
        if v > avg:
            over_avg += 1

    values.sort(reverse=True)
    print(f"총 입력 개수: {len(values)}개")
    print(f"최댓값: {max_val} / 최솟값: {min_val}")
    print(f"평균값: {round(avg, 2)}")
    print(f"임계값 초과 개수: {over_count}개")
    print(f"평균 초과 개수: {over_avg}개")
    print(f"상위 3개 값: {values[:3]}")

```

-
- ▶ 측정값: 85
 - ▶ 측정값: 120
 - ▶ ☐ 임계값(100) 초과! 현재까지 초과 1회
 - ▶ 측정값: 60
 - ▶ 측정값: q
 - ▶ -----
 - ▶ 총 입력 개수: 3개
 - ▶ 최댓값: 120.0 / 최솟값: 60.0
 - ▶ 평균값: 88.33
 - ▶ 임계값 초과 개수: 1개

- ▶ 평균 초과 개수: 1개
- ▶ 상위 3개 값: [120.0, 85.0, 60.0]

설비 적용

이 문제의 핵심은 "리스트를 두 번 도는 것"입니다. 평균보다 큰 값의 개수는 평균을 다 구하기 전에는 셀 수 없으므로, 한 번은 합계용, 한 번은 개수 세기용으로 나눠 돌아야 합니다. 데이터를 한 번에 다 처리하려는 욕심이 오히려 코드를 꼬이게 만드는 대표적인 사례입니다. **필요하면 두 번 돌아도 됩니다.**

오류가 안 났는데 결과가 틀린 이유

오늘 찾은 버그를 종류별로 나눠 보면 왜 어려웠는지가 보입니다. 프로그램의 문제는 크게 세 층으로 나뉘는데, **아래로 갈수록 발견하기 어렵습니다.**

층	언제 드러나나	오늘의 예	발견 방법
문법 오류	실행 즉시 멈춤	오늘은 없었음	에디터가 미리 알려줌
실행 오류	그 줄에 도달할 때	ValueError · TypeError	Traceback 맨 아래 줄
논리 오류	드러나지 않음	오늘 발견한 7건 전부	예상 출력과 한 줄씩 대조

가이드가 예상 출력을 **구분선과 이모지까지 똑같이 맞추라고** 요구한 이유가 여기 있습니다. 출력 형식을 맞춰 두면 눈으로 한 줄씩 비교하는 것만으로 논리 오류를 잡을 수 있기 때문입니다. 형식이 다르면 값이 틀렸는지 형식이 다른 건지 구분이 안 됩니다.

주의

실무에서는 이 대조를 사람이 아니라 코드가 합니다. **테스트 코드**라고 부르는데, "입력이 이것이면 출력은 이것이어야 한다"를 미리 적어두고 자동으로 확인시키는 방식입니다. 지금 손으로 하는 대조가 바로 그 원형입니다. 예상 출력을 먼저 적어두고 코드를 짜는 습관은 나중에 그대로 테스트 작성 능력이 됩니다.

STEP 10

종합 실습 3 — 교대조 센서 경고 로그 분석

집합 연산 + 튜플 정렬 · 아직 안 푼 문제

아직 손대지 않은 문제입니다. 정답 코드를 먼저 보기보다 **어떤 개념을 어디에 쓸지 지도부터** 그려 보는 편이 낫습니다. 가이드의 표현대로 "이거 무슨 개념 써야 하지?"를 먼저 떠올리는 훈련이니까요. 아래 표에서 왼쪽만 보고 오른쪽을 가린 채 스스로 답해 본 뒤 확인하는 방식을 권합니다.

주어진 데이터 — 리스트인 것에 주목

```
morning    = ["TZ_11", "TZ_13", "TZ_11", "TZ_15", "TZ_13", "TZ_11", "TZ_11", "TZ_17"]
afternoon  = ["TZ_13", "TZ_15", "TZ_13", "TZ_19", "TZ_15", "TZ_21", "TZ_13", "TZ_15"]
```

요구사항	쓸 개념	한 줄 힌트
1. 조별 고유 센서 수 + 목록	<code>set()</code> · <code>len()</code> · <code>sorted()</code>	중복 제거 → 세기 → 정렬
2. 두 조 공통 센서	<code>&</code> 또는 <code>.intersection()</code>	교집합
3. 조별 전용 센서	<code>-</code> 또는 <code>.difference()</code>	방향 두 번
4. 전체 경고 센서	<code> </code> 또는 <code>.union()</code>	합집합
5. 센서별 총 횟수 순위	<code>.count()</code> · 튜플 · <code>sorted(reverse=True)</code>	(횟수, 이름) 순서가 핵심
6. 최다 경고 센서	정렬 결과의 [0]	이미 정렬했으니 첫 번째

개념

설계의 갈림길이 하나 있습니다. **원본이 리스트라는 점**입니다. 셋으로 바꾸면 종류는 알 수 있지만 **횟수 정보가 사라집니다**. 요구사항 5번은 횟수를 세야 하므로, 셋은 "센서 종류를 뽑는 용도"로만 쓰고 **횟수는 원본 리스트에서 `.count()`로 세야 합니다**. 즉 두 자료형을 목적에 따라 나눠 쓰는 문제입니다. 이 판단을 스스로 내릴 수 있으면 이 실습은 절반 이상 푼 것입니다.

요구사항 5번 — 이 문제의 핵심

가장 어려운 부분입니다. 힌트가 두 개 있는데 둘 다 오늘 배운 내용입니다.

첫째, (횟수, 센서명) 순서로 튜플을 만든다는 점. STEP 3에서 배운 "정렬 기준을 맨 앞에" 원칙입니다. (센서명, 횟수) 순으로 답으면 이름순으로 정렬되어 버립니다.

둘째, `sorted(리스트, reverse=True)` 하나면 정렬이 끝난다는 점. 별도의 정렬 기준 지정(`key=`)이 필요 없습니다.

요구사항 5 — 순위 산출

```
counts = []
for s in all_set:
    total = morning.count(s) + afternoon.count(s)
    counts.append((total, s))          # (횟수, 이름) 순서!
```

```
counts.sort(reverse=True)
```

```
for rank, (cnt, name) in enumerate(counts, start=1):
    print(f" {rank}위: {name} - {cnt}회")
```

```
▶ 1위: TZ_13 - 5회
▶ 2위: TZ_15 - 4회
▶ 3위: TZ_11 - 4회
▶ 4위: TZ_21 - 1회
▶ 5위: TZ_19 - 1회
▶ 6위: TZ_17 - 1회
```

주의

2위와 3위가 **둘 다 4회**입니다. 여기서 TZ_15가 TZ_11보다 앞에 오는 이유는, 첫 값이 같을 때 파이썬이 두 번째 값 (이름)을 비교하는데 `reverse=True`라 **이름도 내림차순**이 되기 때문입니다. `"TZ_15" > "TZ_11"`이므로 TZ_15가 먼저입니다. 가이드의 예상 출력이 정확히 이 순서인 것은 우연이 아니라 이 규칙의 결과입니다. 동점 순서가 예상과 다르다면 이 규칙을 떠올려 보세요.

전체 정답 코드

완성본 — 실행 검증 완료

```
morning = ["TZ_11", "TZ_13", "TZ_11", "TZ_15", "TZ_13", "TZ_11", "TZ_11", "TZ_17"]
afternoon = ["TZ_13", "TZ_15", "TZ_13", "TZ_19", "TZ_15", "TZ_21", "TZ_13", "TZ_15"]
```

```
m_set = set(morning)
a_set = set(afternoon)
```

```
print("=== 교대조 센서 경고 로그 분석 ===")
print(f"오전조 고유 센서 {len(m_set)}종: {sorted(m_set)}")
print(f"오후조 고유 센서 {len(a_set)}종: {sorted(a_set)}")
print("-" * 40)
```

```

print(f"양 교대조 공통 경고 센서: {sorted(m_set & a_set)}")
print(f"오전조 전용: {sorted(m_set - a_set)}")
print(f"오후조 전용: {sorted(a_set - m_set)}")

all_set = m_set | a_set
print(f"전체 경고 센서 {len(all_set)}종: {sorted(all_set)}")
print("-" * 40)

counts = []
for s in all_set:
    counts.append((morning.count(s) + afternoon.count(s), s))
counts.sort(reverse=True)

print("경고 발생 횟수 순위:")
for rank, (cnt, name) in enumerate(counts, start=1):
    print(f"  {rank}위: {name} - {cnt}회")
print("-" * 40)

top_cnt, top_name = counts[0]          # 튜플 언패킹
print(f"최다 경고 센서: {top_name} ({top_cnt}회) → 우선 점검 필요")

# 도전) 3회 이상만 집중 관리 대상
focus = []
for cnt, name in counts:
    if cnt >= 3:
        focus.append(name)
focus.sort()
print(f"집중 관리 대상: {focus}")

```

```

▶ === 교대조 센서 경고 로그 분석 ===
▶ 오전조 고유 센서 4종: ['TZ_11', 'TZ_13', 'TZ_15', 'TZ_17']
▶ 오후조 고유 센서 4종: ['TZ_13', 'TZ_15', 'TZ_19', 'TZ_21']
▶ -----
▶ 양 교대조 공통 경고 센서: ['TZ_13', 'TZ_15']
▶ 오전조 전용: ['TZ_11', 'TZ_17']
▶ 오후조 전용: ['TZ_19', 'TZ_21']
▶ 전체 경고 센서 6종: ['TZ_11', 'TZ_13', 'TZ_15', 'TZ_17', 'TZ_19', 'TZ_21']
▶ -----
▶ 경고 발생 횟수 순위:
▶   1위: TZ_13 - 5회
▶   2위: TZ_15 - 4회
▶   3위: TZ_11 - 4회
▶   4위: TZ_21 - 1회
▶   5위: TZ_19 - 1회

```

- ▶ 6위: TZ_17 - 1회
- ▶ -----
- ▶ 최다 경고 센서: TZ_13 (5회) → 우선 점검 필요
- ▶ 집중 관리 대상: ['TZ_11', 'TZ_13', 'TZ_15']

설비 적용

이 실습이 만들어내는 결론이 곧 정비 계획입니다. 공통 경고 센서(TZ_13·TZ_15)는 교대와 무관하게 문제가 있으니 **설비 자체를 봐야** 하고, **한쪽 조에만 나타나는 센서**는 작업 방식이나 그 시간대의 부하 차이를 의심해야 합니다. 같은 집합 연산이라도 **어느 축으로 나누느냐**에 따라 완전히 다른 인사이트가 나옵니다. 교대조 대신 라인별·요일별·계절별로 바꿔 보면 각각 다른 이야기가 보입니다.

오늘 배운 것이 어디에 쓰였나

종합 실습 세 문제에 오늘과 지난 회차의 개념이 어떻게 분포했는지 정리했습니다. **한 문제에 이렇게 많이 섞여 있었다**는 사실 자체가 이 실습의 목적입니다.

개념	배운 날	실습 1	실습 2	실습 3
튜플 리스트 + 언패킹	8~9일차	●		●
enumerate	7일차	●		●
if / elif / else	7일차	●	●	●
while + break	8일차		●	
누적 변수 (+=)	3·7일차	●	●	●
.append() · .sort()	6일차	●	●	●
입력 · 형변환	4일차		●	
f-string 서식	5일차	●	●	●
셋 · 집합 연산	9일차			●
sorted + 슬라이싱	9일차	●	●	●

응용

세 문제를 다 푼 뒤 여유가 있다면 데이터를 바꿔 다시 돌려보는 것이 가장 효율적인 복습입니다. 예를 들어 실습 3의 두 리스트를 **오전/오후 대신 월요일/금요일**로 바꾸면 같은 코드가 요일별 분석이 됩니다. 코드는 한 글자도 안 바뀌는데 결론이 달라지는 경험을 하면, 반복문과 집합 연산을 "외운 문법"이 아니라 **도구**로 인식하게 됩니다.

같은 코드, 다른 축 — 확장해 보기

위 응용 콜아웃에서 말한 "축 바꾸기"를 실제로 해보면 이렇습니다. 데이터 이름과 출력 문구만 바꾸고 **로직은 한 줄도 달라지지 않습니다**.

축만 바꾼 코드

```
# 축을 요일로 바꾼 버전 — 로직은 실습 3과 완전히 동일
monday = ["TZ_11", "TZ_13", "TZ_11", "TZ_15", "TZ_13"]
friday = ["TZ_13", "TZ_15", "TZ_19", "TZ_15", "TZ_21"]

m_set, f_set = set(monday), set(friday)

print("두 요일 공통:", sorted(m_set & f_set))
print("월요일 전용:", sorted(m_set - f_set))
print("금요일 전용:", sorted(f_set - m_set))
```

- ▶ 두 요일 공통: ['TZ_13', 'TZ_15']
- ▶ 월요일 전용: ['TZ_11']
- ▶ 금요일 전용: ['TZ_19', 'TZ_21']

축을 바꾸면 **묻는 질문 자체가 달라집니다**. 같은 세 줄이 각각 다른 현장 판단으로 이어집니다.

비교 축	공통이 크면	한쪽만 크면
오전 / 오후	설비 자체의 문제 — 교대와 무관	작업 방식·시간대 부하 차이 의심
어제 / 오늘	조치가 안 먹혔다 — 지속 이상	신규 발생 또는 해소
A라인 / B라인	공용 부품·환경 요인	해당 라인 고유 문제
평일 / 주말	가동률과 무관한 상시 이상	부하량에 따른 이상

개념

가이드 마지막 문장이 이 실습의 진짜 목표입니다 — **"다 풀면 왜 이렇게 짰는지 옆 사람한테 설명해보기. 설명되면 진짜 아는 것."** 코드를 짜는 것과 설명하는 것은 다른 능력이고, 설명하려면 각 줄의 **이유**를 알아야 합니다. 면접에서 프로젝트 설명을 요구받을 때 필요한 것도 정확히 이 능력입니다.

치트시트

오늘 배운 것 한 장 요약

튜플 — 다루기

	문법	하는 일	반환
내장함수	<code>len(t)</code>	전체 개수	정수
메서드	<code>t.count(x)</code>	x가 몇 번 나오나	정수 (없으면 0)
메서드	<code>t.index(x)</code>	x가 처음 나온 위치	정수 (없으면 ValueError)
문법	<code>t[0] · t[-1]</code>	인덱싱	값 하나
문법	<code>t[1:4]</code>	슬라이싱	튜플 (끝 번호 미포함)
문법	<code>a, b = t</code>	언패킹	변수 개수 = 값 개수
문법	<code>t[2][0]</code>	중첩 튜플 접근	안쪽 값

튜플 리스트 — 오늘의 주력 패턴

패턴	코드	언제
기본 순회	<code>for name, value in sensors:</code>	센서마다 같은 처리
번호 붙이기	<code>for i, (n, t, v) in enumerate(s, start=1):</code>	리포트 출력
조건 필터	<code>if value > LIMIT: result.append(name)</code>	경고 대상 추리기
정렬	<code>sorted(sensors, reverse=True)</code>	순위표 · 원본 보존
상위 N	<code>sorted(s, reverse=True)[:3]</code>	Top 3
2단 언패킹	<code>for n, v, pos in s: → x, y = pos</code>	중첩 튜플 처리

셋 — 만들기과 다루기

	문법	하는 일	주의
생성	<code>{"a", "b"}</code>	값을 직접 나열	리스트를 감싸면 오류

	문법	하는 일	주의
생성	<code>set(리스트)</code>	중복 제거	가장 자주 쓰는 형태
생성	<code>set()</code>	빈 셋	{ }는 딕셔너리!
메서드	<code>s.add(x)</code>	값 추가	중복은 무시 · None 반환
연산자	<code>x in s</code>	포함 확인	리스트보다 압도적으로 빠름
내장함수	<code>len(s)</code>	종류 수	중복 제거된 개수
내장함수	<code>sorted(s)</code>	정렬	결과는 리스트

집합 연산 — 두 목록 비교 3종

질문	연산	코드	예시 해석
전체는?	합집합	<code>a b</code> · <code>a.union(b)</code>	공장 전체 센서 종류
공통은?	교집합	<code>a & b</code> · <code>a.intersection(b)</code>	이틀 연속 경고 = 지속 이상
새로 생긴 건?	차집합	<code>today - yesterday</code>	신규 이상
없어진 건?	차집합	<code>yesterday - today</code>	해소된 이상

개념

셋은 원본을 바꾸지 않습니다. `a.union(b)`도 `a | b`도 새 셋을 돌려줄 뿐 `a`는 그대로입니다. 결과가 필요하면 반드시 변수에 담으세요. 반대로 `a |= b`, `a &= b`, `a -= b`는 원본을 직접 바꿉니다 — 리스트의 `+=`와 같은 발상입니다.

오류·함정 사전

한 번씩은 반드시 겪는 것들

B-1 · 오늘 실제로 겪은 오류

✗ 이렇게 하면	무슨 일이	✓ 바로잡기
<code>lst[:3].sort(reverse=True)</code>	None 출력 + 정렬 순서 거꾸로	<code>sorted(lst, reverse=True)[:3]</code>
<code>min_val = 0</code>	양수 데이터에선 최솟값이 영원히 0	<code>min_val = values[0]</code>
elif 안에만 append	임계값 초과 데이터가 통계에서 누락	append 먼저, 경고 판정은 별도 if
peak 갱신을 if 안에	위험 설비 중에서만 최댓값을 찾음	갱신 코드를 if/elif 바깥으로
append만 하고 sort 안 함	목록 순서가 예상 출력과 다름	<code>lst.sort()</code> 또는 <code>sorted(lst)</code>
LIMIT 정의 후 100 하드코딩	기준 바뀌도 동작이 안 바뀜	선언한 상수를 코드에서 실제로 사용

B-2 · 튜플 함정

✗ 이렇게 하면	무슨 일이	✓ 바로잡기
<code>sensor = (78)</code>	int가 됨 — 괄호는 계산용으로 해석	<code>sensor = (78,)</code>
<code>sensor[1] = 80</code>	<code>TypeError: object does not support item assignment</code>	새 튜플을 만들거나 리스트를 쓴다
<code>t.append(4)</code>	<code>AttributeError</code> — 튜플엔 append가 없다	바뀔 데이터면 애초에 리스트로
<code>t.index("없는값")</code>	<code>ValueError: x not in tuple</code>	if "값" in t: <code>t.index("값")</code>
<code>a, b, c, d = (1, 2, 3)</code>	not enough values to unpack (expected 4, got 3)	변수 개수 = 값 개수로 맞춘다
for i, n, t in enumerate(s)	enumerate는 2개만 준다 → 오류	for i, (n, t) in enumerate(s)
<code>t.len()</code>	<code>AttributeError</code> — len은 메서드가 아니다	<code>len(t)</code>

B-3 · 셋 함정

✗ 이렇게 하면	무슨 일이	✓ 바로잡기
<code>empty = {}</code>	dict가 만들어짐 — add 호출 시 오류	<code>empty = set()</code>
<code>unique = {logs}</code>	<code>TypeError: unhashable type: 'list'</code>	<code>unique = set(logs)</code>
<code>unique[0]</code>	<code>TypeError: 'set' object is not subscriptable</code>	<code>sorted(unique)[0]</code>
<code>sorted = sorted(alerts)</code>	내장 함수를 덮어씀 → 이후 'list' object is not callable	<code>result = sorted(alerts)</code>
<code>alerts = alerts.add("S05")</code>	add는 None 반환 → 셋이 통째로 사라짐	<code>alerts.add("S05")</code>
a - b 와 b - a를 같다고 가정	차집합은 방향에 따라 결과가 완전히 다름	어느 쪽이 기준인지 먼저 정한다

주의

B-3의 네 번째 항목은 **강사님 예제 코드에서도 나온** 패턴입니다. 동작하는 코드라 그 자리에서는 문제가 없지만, 같은 파일 아래쪽에서 `sorted()`를 다시 부르는 순간 터집니다. 변수 이름을 지을 때 **파란색·보라색으로 강조되는 단어(에디터가 내장 이름으로 인식한 것)**는 피하는 것이 안전합니다.

실습 하이라이트 & 다음 시간 예고

오늘 만든 것과 앞으로 만들 것

C-1 · 오늘 작성한 파일

파일	내용	상태
Python/14_tuple.py	len·count·index · 튜플 리스트 · for 언패킹 · sorted	수업 코드
Python/15_set.py	set 생성 · add · in · 집합 연산 3종	수업 코드
Practice/06_01_example.py	교안 실습 1~6	제출 · STEP 7 점검
Practice/16_practice_answer_01.py	종합 실습 1 — 모니터링 리포트	제출 · 버그 2건
Practice/16_practice_answer_02.py	종합 실습 2 — 실시간 입력 시스템	제출 · 버그 5건
Practice/16_practice_03.py	종합 실습 3 — 교대조 로그 분석	미제출 · STEP 10 참고

C-2 · 오늘 안 나간 부분

교안 06_01은 튜플과 셋 두 파트로 끝나므로 오늘로 완주했습니다. 다만 교안 도입부(4~5쪽)에 "뭉는 튜플, 거르는 셋, 찾는 딕셔너리"라는 문장이 나오는데, 딕셔너리는 오늘 언급만 되고 진도로는 나가지 않았습니다. 이 책에서도 CHAPTER 0의 비교표에 회색으로만 넣어 두었습니다.

정렬 기준을 지정하는 `key=` 옵션도 마찬가지입니다. 강사님이 화면으로 보여주시면서 "함수를 배운 뒤에 다시 다루겠다"고 미뤄 두셨습니다. 그때까지는 정렬 기준을 튜플 맨 앞에 두는 방식으로 충분합니다.

C-3 · 다음 시간 예고

다음 주제	한 줄 미리보기
딕셔너리	<code>{"이름": 값}</code> — 번호가 아니라 이름표로 찾는 자료형. 셋과 같은 중괄호를 쓰지만 키:값 쌍이 들어간다
딕셔너리 순회	<code>for k, v in d.items():</code> — 오늘의 튜플 언패킹이 그대로 재등장한다
함수 (def)	반복되는 코드를 이름 붙여 재사용. <code>key=</code> 정렬도 이때 풀린다

다음 주제	한 줄 미리보기
pandas	오늘의 튜플 리스트가 표(DataFrame)가 되고, 집합 연산이 두 표 비교로 확장된다

연결

오늘로 **기본 자료형 4형제 중 3개**를 끝냈습니다 — 리스트·튜플·셋. 마지막 딕셔너리까지 배우면 파이썬으로 데이터를 담는 그릇은 전부 손에 넣는 셈입니다. 그 다음이 함수이고, 함수까지 익히면 pandas로 넘어갈 준비가 끝납니다. 지금 있는 지점이 **파이썬 기초의 마지막 구간**이라는 것을 알고 가면 동기 유지에 도움이 됩니다.

오늘의 성취와 다음 한 걸음

묶고, 걸러서, 비교한다

오늘 배운 것을 한 문장으로 줄이면 이렇습니다 — **짜지어진 데이터는 튜플로 묶어 리스트에 모으고, 반복문으로 처리하고, 중복을 건어내 두 목록을 비교한다.** 이 한 문장이 앞으로 다룰 설비 센서 데이터 처리의 기본 골격입니다.

그리고 오늘은 처음으로 **여러 날 배운 것을 한 문제에 몰아넣는 종합 실습**을 했습니다. 문법 하나하나를 알고 있었는데 막상 조합하려니 막히는 지점이 있었을 겁니다. 그게 정상입니다. 가이드에 적힌 말 그대로 "몰라서 못 푸는 게 아니라 안 떠올라서 못 푸는 것"이고, 안 떠오르는 건 반복으로만 해결됩니다.

발견한 버그 7건 — 요약

종합 실습 1·2에서 나온 버그 일곱 건 중 **오류 메시지가 뜬 것은 단 하나도 없었습니다.** 코드는 멀쩡히 돌고 결과만 틀렸습니다. 이 종류의 버그를 잡는 방법은 하나뿐입니다 — **예상 출력과 실제 출력을 한 줄씩 대조하는 것.** 가이드가 예상 출력을 그렇게 자세히 준 이유가 여기 있습니다.

개념

복습 루틴 추천: ① len은 함수, count·index·add는 메서드 — 점 유무로 3초 판별 → ② for name, value in sensors: 를 눈 감고 쓸 수 있을 때까지 → ③ sorted(리스트, reverse=True)[:3]의 순서를 손으로 써보며 확인 → ④ 빈 리스트 [], 빈 튜플 (), 빈 셋 set() 세 개를 한 세트로 암기 → ⑤ 종합 실습 3을 STEP 10을 덮고 처음부터 직접 풀어보기. 특히 ⑤는 오늘 안 푼 문제이므로 **제일 먼저** 하는 것을 권합니다.